Book 3: Applications & Tokenomics

Wallets, First-Party Privacy Modules (xUSD CDP, AI Usage Tickets, Stealth Invoices), DEX/CEX integrations, SER Tokenomics, Presale, and Creator Launchpad.

- Chapter 10: Wallets, Scanning & Key Recovery
- Chapter 11: First-Party Privacy Modules
- Chapter 12: Ecosystem, DEX & CEX Integrations
- Chapter 13: SER Tokenomics, Supply & Burns
- Chapter 14: Presale Architecture & Vesting
- Chapter 15: Launchpad Mechanics & Creator Playbook

Chapter 10: Wallets, Scanning & Key Recovery

Chapter 10: Wallets, Scanning & Key Recovery

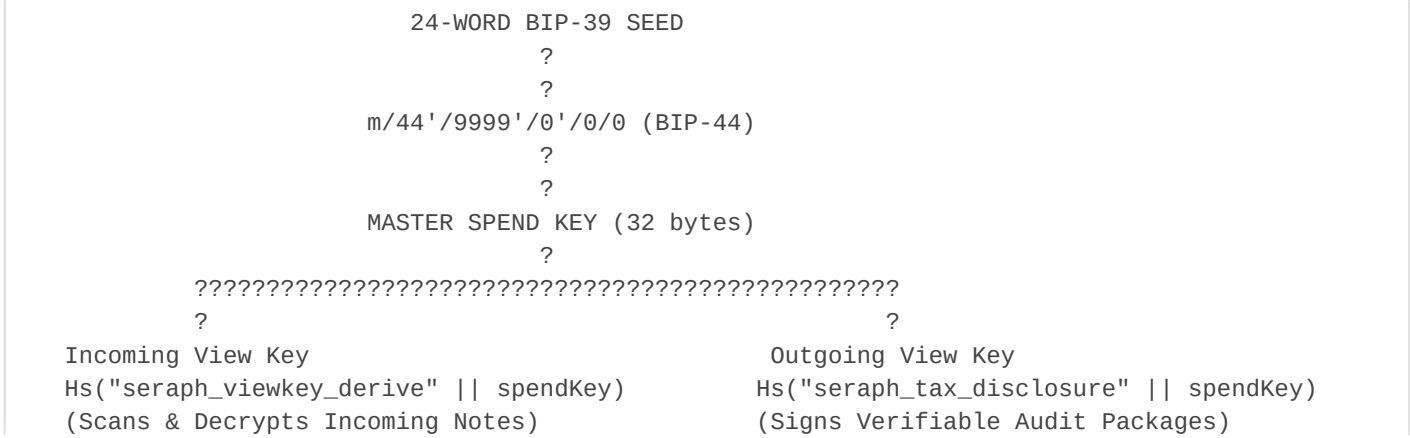
10.1 The Official Seraph Wallet Ecosystem

The Seraph wallet ecosystem provides non-custodial key management and privacy execution across desktop, mobile, and web interfaces:

- **Desktop Wallet (macOS, Linux, Windows)**: Full-featured native application with local zero-knowledge prover acceleration and optional integrated full-node synchronization.
- **Web Wallet & Browser Extension**: Lightweight client integrated directly into modern browsers, utilizing WASM and WebGPU shaders for client-side proof generation.
- **Mobile Wallet (iOS, Android)**: Fast, touch-optimized wallet with biometric key protection and delegated scan-key processing.

10.2 Key Hierarchy & Derivation

Seraph derives all cryptographic keys deterministically from a single standard 24-word BIP-39 mnemonic seed phrase:



```
          ?                                       ?
          ?                                       ?
  Stealth Public Key                      Compliance Signature
  Hs("seraph_pubkey_derive" || spendKey)  Tax / Audit Verification
```

10.3 Client-Side Note Scanning (`scanNotesForOwner`)

Because note commitments on the blockchain are cryptographically blinded, a wallet must scan new ledger blocks to identify which notes belong to the user.

In Seraph, this is accomplished via **trial decryption** using the wallet's **Incoming View Key**:

```
// Production Note Scanning Engine (sdk/typescript/src/index.ts)
const scannedNotes = scanNotesForOwner(encryptedNotes, incomingViewKey);
```

The Magic Tag & Decryption Layout:

Every note memo payload is encoded with a 4-byte magic identifier (`NOTE_MAGIC = "SERP"` = `0x53455250`): $$\text{Memo Layout} = [\text{4 bytes "SERP"}] \mathbin{\Vert} [\text{8 bytes Amount BE}] \mathbin{\Vert} [\text{1 byte AssetId}] \mathbin{\Vert} [\text{UTF-8 Memo}]$$

1. The wallet iterates through new note commitments.
2. For each note, the wallet decrypts the ciphertext using its `incomingViewKey` and the note's public `blinding`: $$\text{decrypted} = \text{xorViewKeyMemo}(\mathcal{K}_{\text{in}}, \text{blinding}, \text{encryptedMemo})$$
3. The wallet inspects the first 4 bytes:
   - If `decrypted[0..3] == 0x53455250`, the note **definitely belongs to this wallet**!
   - If the magic does not match, the note belongs to another user and is instantly discarded.
4. For matched notes, the wallet decodes the exact spendable balance, the asset type (`AssetId.Seraph` or `AssetId.Xmr`), and the secret transaction memo.

10.4 Verifiable Tax & Compliance Disclosures

Privacy should not require users to violate financial reporting laws. Seraph enables users to generate mathematical, verifiable compliance disclosure packages signed by their **Outgoing View Key**:

```
// Generate Tax Package (sdk/typescript/src/index.ts)
const disclosure = generateComplianceDisclosure(spentRecords, outgoingViewKey);
```

Disclosure Package Structure:

- `disclosureSummary`: JSON payload enumerating:
  - Consumed nullifiers ($\mathcal{N}_i$).
  - Exact unshielded amounts.
  - Destination public addresses.
  - Timestamps and on-chain transaction hashes.
- `signatureTag`: 32-byte cryptographic HMAC tag: $$\text{signatureTag} = \text{SHA256Tag}(\text{"seraph_tax_disclosure_v1"}, \mathcal{K}_{\text{out}}, \text{summaryBytes})$$

An auditor or tax accountant possessing the user's `outgoingViewKey` can verify the signature and prove that all reported transactions are authentic and comprehensive, without being given spending authority.

10.5 Encrypted Wallet Backups & Paper Recovery

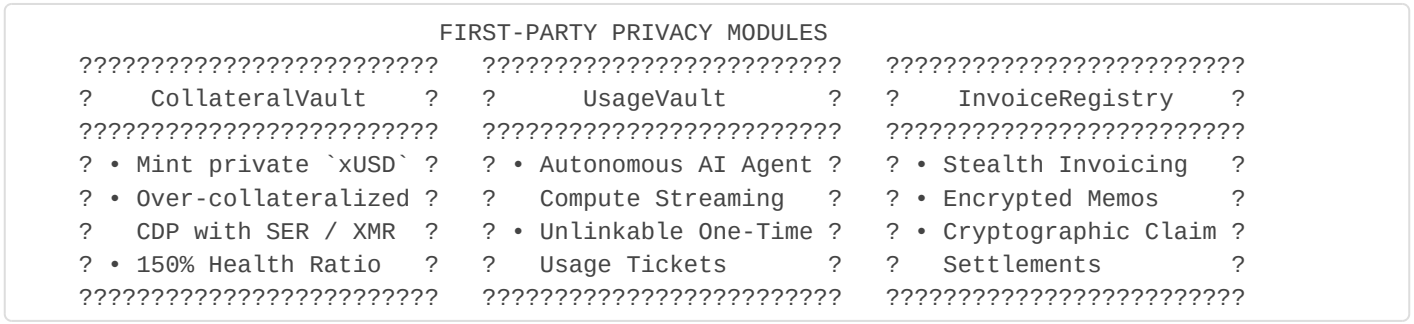
1. **Standard Seed Phrase**: The 24-word seed phrase is sufficient to recover 100% of all past, present, and future funds and transaction history.
2. **Encrypted JSON Vault**: Desktop and web wallets allow exporting an AES-GCM-256 encrypted backup file containing cached Merkle paths, note commitments, and local address labels for instantaneous wallet restore without requiring a full chain rescan.
3. **Emergency Paper Key**: A printable cold-storage certificate containing the master spend key and incoming view key in high-density QR code format.

Chapter 11: First-Party Privacy Modules

Chapter 11: First-Party Privacy Modules

11.1 The Ecosystem of First-Party Privacy Modules

Seraph extends beyond simple token transfers by providing an integrated suite of native, first-party privacy applications deployed directly on the confidential EVMs:



11.2 Private Stablecoins: `xUSD` & `CollateralVault`

Volatility in crypto assets makes everyday commerce difficult. The `CollateralVault` enables users to lock SER or XMR collateral and mint **`xUSD`**—a confidential, decentralized dollar stablecoin.

Mechanics & Invariants:

1. **Minimum Collateralization Ratio (150%)**: $$\text{Collateral Ratio} = \frac{\text{Collateral} \times \text{Price}}{\text{USD}}}{\text{Debt}}{\text{xUSD}}} \ge 150%$$
2. **Deposit & Mint**:

```
function depositAndMint(
    uint256 collateralAmount,
    uint256 mintAmount
) external nonReentrant;
```

Users deposit collateral and mint synthetic `xUSD` directly into their private balances.

3. **Burn & Withdraw**:

```
function burnAndWithdraw(
    uint256 burnAmount,
    uint256 withdrawCollateral
) external nonReentrant;
```

Users burn `xUSD` to unlock their underlying collateral, maintaining the 150% ratio.

4. **Confidential Circulation**: Once minted, `xUSD` can be shielded and transferred through the `PrivacyPool` with zero public record of sender, recipient, or balance.

11.3 Autonomous AI Agent Billing: `UsageVault`

As autonomous AI agents, machine learning inference nodes, and cloud compute clusters transact on-chain, they require high-frequency, low-latency, unlinkable micro-payments without exposing corporate infrastructure budgets.

The `UsageVault` implements **cryptographic one-time usage tickets**:

```
 Client / AI Agent                                    UsageVault
Contract                              AI Compute Provider
        ?                                                    ?
                                                ?
        ? 1. Deposits credits (`depositCredits`)             ?
                                                ?
        ?    Commitment = H(amount, secretPreimage)          ?
                                                ?
        ?????????????????????????????????????????????????>?
                                                ?
        ?                                                    ?
                                                ?
        ? 2. Sends one-time ticket off-chain to Provider    ?
                                                ?
        ?    (ticketNullifier = H(secretPreimage), amount)   ?
                                                ?
        ?
????????????????????????????????????????????????????????????????????????????????????????????????????
???>?
        ?                                                    ?
                                                ?
        ?                                                    ? 3. Redeems ticket
(`redeemUsageTicket`)       ?
        ?                                                    ?    Verifies H(amount, secret) ==
commitment   ?
        ?                                                    ? <
```

```
????????????????????????????????????????????
        ?                                                    ?
                                                ?
        ?                                                    ? 4. Withdraws provider
revenue                 ?
        ?                                                    ?
(`claimEarnings`)                           ?
```

SDK Integration:

```
import { createCreditDeposit, UsageVaultClient } from "@seraph-protocol/sdk";

// 1. Create unlinkable credit ticket
const secretPreimage = randomBytes32();
const ticket = createCreditDeposit(100n * 10n**18n, secretPreimage);

// 2. Deposit into vault
const client = new UsageVaultClient(vaultAddress);
const tx = client.encodeDeposit(ticket.commitment, 100n * 10n**18n);
```

11.4 Stealth Invoicing & Encrypted Messaging: `InvoiceRegistry`

The `InvoiceRegistry` contract solves privacy for merchant invoicing and peer-to-peer business transactions:

1. Invoice Creation

```
function createInvoice(
    bytes32 invoiceId,
    bytes32 stealthRecipient,
    address claimAddr,
    address token,
    uint256 amount,
    bytes calldata encryptedMemo
) external nonReentrant;
```

- The merchant publishes an invoice bound to a stealth destination address.
- The memo (specifying goods, order ID, or service description) is encrypted under the buyer's view key.

2. Settle & Nullify

```
function settleInvoice(
    bytes32 invoiceId,
    bytes32 paymentNullifier
) external nonReentrant;
```

- When the payment is executed, the `paymentNullifier` is permanently recorded.
- Replaying payments across different invoices is strictly prevented (`NullifierAlreadyUsed`).

3. End-to-End Encrypted Messaging

Users can broadcast encrypted messages to any recipient's public key:

```
function sendEncryptedMessage(
    bytes32 recipientPubkey,
    bytes32 ephemeralPubkey,
    bytes calldata ciphertext
) external;
```

This enables native on-chain communication for trade negotiations, OTC settlement confirmations, and payment notices.

Chapter 12: Ecosystem, DEX & CEX Integrations

Chapter 12: Ecosystem, DEX & CEX Integrations

12.1 Overview: Integrating Privacy with Global Liquidity

Decentralized finance (DeFi) liquidity and centralized exchange (CEX) listings require standard, predictable ERC-20 token interactions. However, protocols featuring transfer taxes or non-standard balance logic frequently experience routing failures, slippage mismatches, or unexpected fees during multi-hop DEX swaps.

Seraph solves this through a dedicated **Tax Exemption Registry** and standardized integration interfaces designed specifically for automated market makers (AMMs) and institutional custodians.

12.2 Decentralized Exchanges (Uniswap v3 & Aerodrome)

The Multi-Hop Double-Tax Problem

On a standard DEX, a swap from SER to USDC often routes through an intermediary pool: $$\text{User} \xrightarrow{\text{Swap}} \text{Pool}_{\text{SER/WETH}} \xrightarrow{\text{Internal Transfer}} \text{Router} \xrightarrow{\text{Swap}} \text{Pool}_{\text{WETH/USDC}} \xrightarrow{} \text{User}$$ If a 1.00% sell tax is assessed on every hop, the user would be taxed twice (or the DEX router would fail due to received amount mismatches).

The Tax Exemption Mapping (`taxExempt`)

To ensure seamless integration with Uniswap v3, Aerodrome, Curve, and Balancer, `SERAPH.sol` includes a dynamic exemption mapping:

```
// contracts/core/SERAPH.sol
mapping(address => bool) public taxExempt;
event TaxExemptionUpdated(address indexed account, bool isExempt);

function setTaxExempt(address account, bool isExempt) external onlyTreasury {
    taxExempt[account] = isExempt;
    emit TaxExemptionUpdated(account, isExempt);
}
```

In the internal `_update` transfer hook:

```
bool isSell = from != address(0) && to != address(0) && from != address(this) && to !=
address(this)
    && from != presale && to != presale && from != treasury && to != treasury
    && from != privacyPool && to != privacyPool
    && !taxExempt[from] && !taxExempt[to];
```

Whitelisting DEX Contracts:

1. When a new Uniswap v3 pool or Aerodrome gauge is deployed, the Seraph Treasury (governed by the Protocol DAO) invokes:
   ```
   seraphToken.setTaxExempt(uniswapV3PoolAddress, true);
   seraphToken.setTaxExempt(universalRouterAddress, true);
   ```
2. Once whitelisted, transfers between the router, pools, and liquidity providers execute with **0% tax**.
3. Sells initiated by end users on public markets continue to contribute to the deflationary burn and protocol treasury.

12.3 Centralized Exchanges (CEX) Custody & Operations

Centralized exchanges (such as Coinbase, Binance, OKX, and Kraken) utilize hot-and-cold wallet architectures where funds are swept internally between omnibus deposit addresses, cold storage vaults, and hot withdrawal wallets.

Custody Whitelisting Workflow:

1. Prior to listing, the exchange provides their omnibus hot/cold wallet addresses to the Seraph Treasury.
2. The Treasury adds these custody addresses to `taxExempt`:
   ```
   seraphToken.setTaxExempt(cexOmnibusWallet, true);
   ```
3. **Benefits**:
   - Internal exchange balance sweeps incur **0% tax**.
   - User deposits and withdrawals match exact balances to the single wei.
   - Zero reconciliation discrepancy for exchange accounting databases.

12.4 Autonomous AI Agent Payment SDK Integration

Autonomous AI agents transacting across decentralized networks can integrate `@seraph-protocol/sdk` with zero external dependencies:

```
import {
  deriveWalletKeys,
  computeNoteCommitment,
  UsageVaultClient,
  createCreditDeposit,
  AssetId,
} from "@seraph-protocol/sdk";

// Initialize autonomous agent wallet
const agentSpendKey = Buffer.from(process.env.AGENT_SPEND_KEY!, "hex");
const keys = deriveWalletKeys(agentSpendKey);

// Prepare usage credit deposit for compute streaming
const computeCost = 10n * 10n**18n; // 10 SER
const secretPreimage = crypto.randomBytes(32);
const ticket = createCreditDeposit(computeCost, secretPreimage);

// Submit on-chain via client
const vaultClient = new UsageVaultClient(USAGE_VAULT_ADDRESS);
const tx = vaultClient.encodeDeposit(ticket.commitment, computeCost);
```

12.5 Standardized JSON-RPC & WebSocket Streaming

Seraph nodes expose standard JSON-RPC 2.0 endpoints for ecosystem tooling:

Custom Privacy RPC Methods:

- `seraph_getLastRoot()`: Returns the latest Merkle root of the active commitment tree.
- `seraph_isKnownRoot(root)`: Verifies if a given 32-byte root exists within the 100-root ring buffer.
- `seraph_getNotesByRange(startLeaf, endLeaf)`: Fetches encrypted note memo payloads for client-side scanning.
- `seraph_estimateGasVoucher(feeToken, gasLimit)`: Calculates the required SER or XMR fee voucher amount for gasless execution.

WebSocket Subscriptions:

Clients can listen to live on-chain privacy events via WebSocket:

```
const ws = new WebSocket("wss://rpc.seraphprotocol.tech/ws");
ws.send(JSON.stringify({
  jsonrpc: "2.0",
  id: 1,
  method: "eth_subscribe",
  params: ["logs", { address: PRIVACY_POOL_ADDRESS }]
}));
```

This allows wallets to detect incoming payments in real time (< 1 second latency).

Chapter 13: SER Tokenomics, Supply & Burns

Chapter 13: SER Tokenomics, Supply & Burns

13.1 Fixed Total Supply & Hard Cap

The native utility and governance token of Seraph Protocol is **SER**. The token is governed by a strictly immutable supply cap enforced at the smart contract level:

$$\text{MAX_SUPPLY} = 100,000,000 \times 10^{18} \text{ wei } (100\text{ Million SER})$$

In `contracts/core/SERAPH.sol`:

```
uint256 public constant MAX_SUPPLY = 100_000_000 * 1e18;

function mint(address to, uint256 amount) external onlyMinter {
    require(totalSupply() + amount <= MAX_SUPPLY, "SeraphToken: max supply exceeded");
    _mint(to, amount);
}
```

No administrative role, multisig, or governance proposal can ever expand supply beyond 100M tokens.

13.2 Genesis Token Allocation

The 100,000,000 SER token allocation is structured to balance community participation, decentralized ecosystem bootstrapping, and long-term protocol security:

```
                        100M SER ALLOCATION
   ??????????????????????????????????????????????????????????????????????
   ? [??????????????????????] Presale & Community Tiers   (30.0%)   ?
   ? [??????????????????]       Protocol Treasury & DAO     (25.0%)   ?
   ? [????????????]             Liquidity & DEX Bootstrapping(20.0%)  ?
   ? [?????????]                Core Contributors & Team    (15.0%)   ?
   ? [??????]                   Ecosystem Grants & Research (10.0%)   ?
   ??????????????????????????????????????????????????????????????????????
```

Category	Allocation	Token Amount	Vesting Schedule
Presale (Strategic, Private, Public)	30.0%	30,000,000 SER	Strategic: 25% TGE, 14d cliff, 12m linear Private: 20% TGE, 14d cliff, 12m linear Public: 100% TGE unlock
Protocol Treasury & DAO	25.0%	25,000,000 SER	Multi-year DAO governance release
Liquidity & DEX Bootstrapping	20.0%	20,000,000 SER	100% unlocked at TGE for AMM depth
Core Contributors & Team	15.0%	15,000,000 SER	12-month cliff, 36-month linear vesting
Ecosystem Grants & Research	10.0%	10,000,000 SER	Milestone-gated via DAO proposals

---

13.3 Multi-Faceted Token Utility

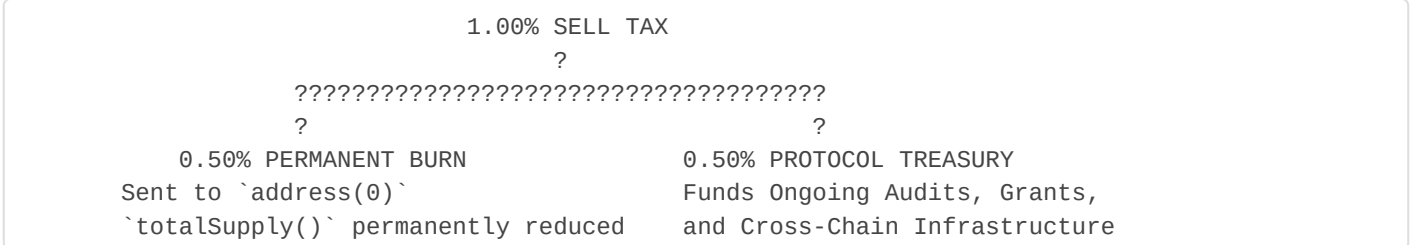
SER powers all economic activity across the Privacy Continuum:

1. **Gas Fee Settlement**: Primary gas asset for executing contracts and state transitions on Seraph EVM.
2. **Privacy Pool Collateral**: Staked or locked in `PrivacyPool` and `CollateralVault` to mint confidential synthetic `xUSD`.
3. **Decentralized Governance**: Token holders vote on protocol parameters, precompile additions, and treasury grants using anonymous zero-knowledge ballots.
4. **Relayer & Sequencer Staking**: Relayers bond SER to earn routing fees on the Cross-EVM Bridge and participate in block sequencing.

13.4 The Deflationary Sell Tax & Burn Engine

To ensure long-term value accrual and discourage predatory mercenary dumping, `SERAPH.sol` incorporates an automated sell-side fee mechanism:

- **Sell Tax Rate**: 1.00% (`SELL_TAX_BPS = 100`).
- **Burn Split**: 50% permanently burned (`BURN_BPS = 5000`).
- **Treasury Split**: 50% directed to the Protocol Treasury (`toTreasury = tax - burned`, official address: `0xa287FC2B8f89023eF1c82af09aee351cCdDa8a76`).



Invariant & Mathematical Proofs

Our test suite rigorously verifies two inviolable economic invariants across random fuzz actions (`PermitTaxAndPresaleStress.t.sol`):

1. **Supply Conservation**: $$\text{totalSupply}() + \text{totalBurned} == \text{totalMinted}$$
2. **Absolute Ceiling**: $$\text{totalSupply}() \le \text{MAX_SUPPLY}$$

Zero Tax on Shielding & Unshielding:

Transfers into and out of the `PrivacyPool` are strictly exempt from the sell tax:

- `shield()`: 0% Tax.
- `privateTransfer()`: 0% Tax.
- `unshield()`: 0% Tax. This guarantees that privacy preservation is never penalized with protocol friction.

Chapter 14: Presale Architecture & Vesting

Chapter 14: Presale Architecture & Vesting

14.1 Presale Philosophy & Structural Design

The Seraph presale (`Presale.sol`) is designed to distribute early protocol ownership transparently across three tiers while preventing post-TGE dumping through mathematical vesting schedules.

The presale contract is deployed and verified on Base Sepolia (`0x2c4156e4bFE0bE6A4e683C7Be521E1cC10B798bE`) and supports seamless multi-currency investments in **ETH**, **USDC**, and **USDT**.

14.2 Tier Specifications & Allocation Matrix

Tier ID	Tier Name	Token Price	Allocation Cap	TGE Unlock	Cliff Period	Linear Vesting	Access Gating
Tier 1	**Strategic**	**$0.02**	10,000,000 SER	**25.0%**	14 Days	12 Months	Whitelist / KYC Required
Tier 2	**Private**	**$0.03**	10,000,000 SER	**20.0%**	14 Days	12 Months	Whitelist / KYC Required
Tier 3	**Public**	**$0.04**	10,000,000 SER	**100.0%**	None (0 Days)	None (0 Months)	Open to All (No KYC)

Key Economic Properties:

- **Total Raised Potential**: $$\text{Strategic: } 10\text{M} \times $0.02 = $200,000$$ $$\text{Private: } 10\text{M} \times $0.03 = $300,000$$ $$\text{Public: } 10\text{M} \times $0.04 = $400,000$$

$$\text{Total Target Hard Cap} = $900,000$$
- **Presale Protocol Fee**: A nominal **0.50% fee** (`FEE_BPS = 50`) is deducted on purchases to cover deployment and security audit costs.

14.3 Multi-Currency Payment Engine

Investors can purchase allocations using their preferred asset:

1. **USDC (6 Decimals)**: `buyWithStablecoin(tierId, amount, usdc, referrer)`
2. **USDT (6 Decimals)**: `buyWithStablecoin(tierId, amount, usdt, referrer)`
3. **ETH (18 Decimals)**: `buyWithETH(tierId)` or direct wallet transfer via `receive()` fallback.

Dynamic ETH/USD Pricing:

The presale contract maintains an on-chain price parameter (`ethUsdPrice`) updated by the Treasury to mirror real-time market rates: $$\text{Tokens Purchased} = \frac{\text{ETH Amount} \times \text{ethUsdPrice}}{\text{Tier Price}}$$ Normalizing 6-decimal stablecoins to 18-decimal token precision is handled automatically with full integer precision and zero truncation loss.

14.4 Vesting Mathematics & Claim Lifecycle

For an investor with total allocation A in Tier 1 or Tier 2:

Phase 1: TGE Claim (`claimInitialAllocation`)

At Token Generation Event ($t \ge T_{\text{TGE}}$): $$\text{Initial Claimable} = A \times \frac{\text{UnlockBPS}}{10,000}$$

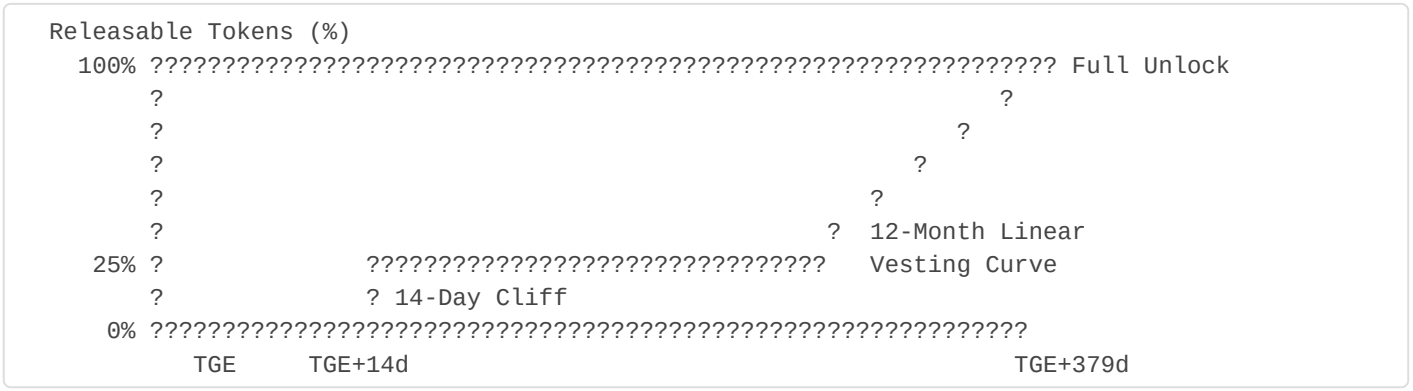
- Strategic investors claim **25%** immediately.
- Private investors claim **20%** immediately.
- Public investors claim **100%** immediately via `claimInitialAllocation()`.

Phase 2: The Cliff Period

For the first 14 days following TGE ($T_{\text{TGE}} \le t < T_{\text{TGE}} + 14 \text{ days}$): $$\text{Vested Amount} = 0$$ No additional tokens are releasable during the cliff.

Phase 3: Continuous Linear Vesting (`claimVested`)

Following the 14-day cliff ($t \ge T_{\text{TGE}} + 14 \text{ days}$): $$\text{Releasable}(t) = \left(A - \text{Initial} \right) \times \min\left(1, \frac{t - (T_{\text{TGE}} + 14 \text{ days})}{365 \text{ days}}\right) - \text{AlreadyReleased}$$



Zero-Dust Guarantee

Our test suite confirms that after $t \ge T_{\text{TGE}} + 379 \text{ days}$: $$\text{v.released} == \text{v.total} \quad (\text{Exact to the single wei})$$ No fractional wei is permanently stranded in the contract.

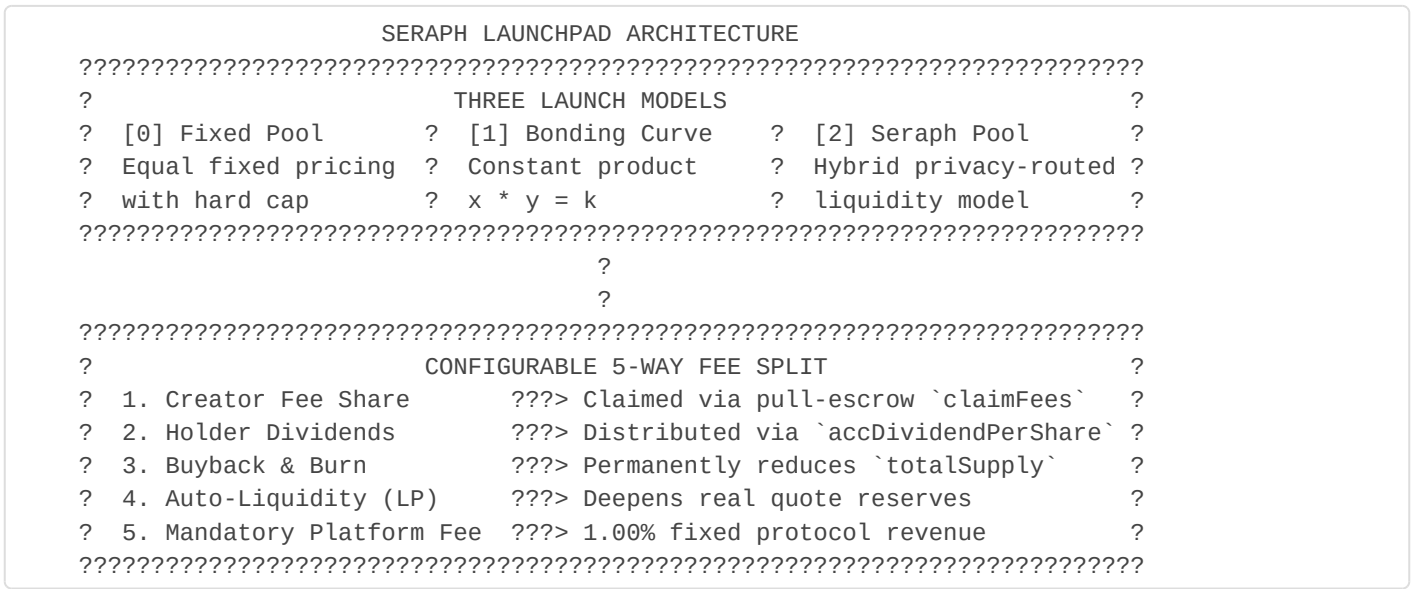
Chapter 15: Launchpad Mechanics & Creator Playbook

Chapter 15: Launchpad Mechanics & Creator Playbook

15.1 Overview: Fair Launches for the Privacy Era

The Seraph Launchpad (`Launchpad.sol`) enables creators, autonomous agents, and decentralized communities to launch privacy-native ERC-20 tokens with customizable economic flywheels, continuous bonding curves, and automated dividend distributions.

The contract is live on Base Sepolia at `0xe0349E4cC431329207e98C5205062776e058A846`.



15.2 The Three Launch Models

1. **Fixed Pool (`Model 0`)**:
   - Fixed token price from launch to graduation.

- Ideal for predictable public token allocations and community airdrop claims.

2. **Bonding Curve (`Model 1`)**:
   - Continuous constant-product automated market maker ($x \cdot y = k$).
   - Token price rises monotonically on every buy and falls on every sell.
   - Phantom virtual liquidity (Q_{phantom}) provides initial price stability without requiring upfront creator capital.
3. **Seraph Pool (`Model 2`)**:
   - Hybrid bonding curve tightly integrated with the `PrivacyPool`.
   - Buyers can opt to shield received tokens directly into private notes in the same transaction.

15.3 The Fee Model, Anti-Spam Launch Fee & 6 UI Presets

The launchpad charges:

- A **flat anti-spam creation fee of 0.001 ETH** (~$2.50 USD), paid at token deployment and routed directly to the Seraph Treasury (`0xa287FC2B8f89023eF1c82af09aee351cCdDa8a76`) to prevent automated bot flood attacks.
- A **mandatory 1.00% Seraph platform fee** on every buy and sell (`PLATFORM_FEE_BPS = 100`).
- A **creator fee of up to 2.50%** (`buyFeeBps`, `sellFeeBps` ≤ 250).

Creators allocate their trading fee revenue across 5 destinations summing to 10,000 basis points (100%):

$$\text{creatorShare} + \text{buybackShare} + \text{dividendsShare} + \text{liquidityShare} + \text{platformShare} = 10,000 \text{ bps}$$

The 6 Creator UI Presets:

1. **Creator-Backed**: 100% of creator fees to creator wallet. Best for artists, builders, and direct venture funding.
2. **Diamond Hands**: 100% of creator fees to holder dividends. Rewards long-term holders with continuous ETH passive yield.
3. **Deflationary**: 100% of creator fees to buyback and burn. Systematically contracts circulating supply on every trade.
4. **Auto-LP**: 100% of creator fees to curve liquidity. Accelerates graduation and builds massive liquidity depth.
5. **Send to Seraph**: 100% of creator fees to the Seraph DAO treasury. Supports protocol development and grant funds.
6. **Custom**: Fully bespoke allocation across all 5 categories.

15.4 Mathematical Dividend Distribution Algorithm

Tokens launched with dividend splits distribute ETH yields proportionally to all token holders without requiring expensive loop iterations across thousands of addresses.

The algorithm uses a global accumulator: $$\text{accDividendPerShare} \mathrel{+}= \frac{\Delta \text{DividendEth} \times 10^{18}}{\text{EligibleTokenSupply}}$$

Per-User Claim Accounting:

For any token holder u with balance B_u: $$\text{Claimable}(u) = \left(B_u \times \text{accDividendPerShare} \right) - \text{rewardDebt}_u$$

Secondary Transfer Conservation:

When holder u transfers A tokens to recipient r on secondary markets:

1. Holder u's unclaimed dividends are settled immediately: $$\text{claimableDividends}_u \mathrel{+}= \left(B_u \times \text{accDividendPerShare} \right) - \text{rewardDebt}_u$$
2. Recipient r's unclaimed dividends are settled: $$\text{claimableDividends}_r \mathrel{+}= \left(B_r \times \text{accDividendPerShare} \right) - \text{rewardDebt}_r$$
3. Balances update: $$B_u \leftarrow B_u - A, \quad B_r \leftarrow B_r + A$$
4. Reward debts update: $$\text{rewardDebt}_u = B_u \times \text{accDividendPerShare}$$ $$\text{rewardDebt}_r = B_r \times \text{accDividendPerShare}$$

Our test suite proves that ping-ponging tokens through 10 sybil addresses never increases total claimable dividends by even a single wei.

15.5 Creator Playbook: Step-by-Step Launch Guide

Step 1: Formulate Token Configuration

Define metadata, ticker, initial supply (minimum 10^{18} wei), fee caps ($\le 2.5\%$), and select a fee preset.

Step 2: Atomic Dev Buy (Optional up to 10%)

Creators can atomically purchase up to 10% (`MAX_DEV_BUY_BPS = 1000`) of the token supply in the exact same transaction as the launch. This prevents front-running snipers from buying before the creator.

Step 3: Broadcast Launch

The creator broadcasts `launch(params)` payable with the required ETH:

```
address tokenAddress = launchpad.launch{value: devBuyEth}(params);
```

Step 4: Community Trading & Fee Escrow

Traders buy and sell on the curve. Fees accumulate in the contract's pull-escrow balance. Creators and the platform claim their ETH anytime via:

```
launchpad.claimFees();
```

Because fees use **pull-escrow** rather than immediate pushes, malicious creator contracts with failing `receive()` fallbacks cannot freeze trading or create honeypots.

Step 5: Graduation Threshold

When real ETH reserves reach the graduation target:

- Trading on the bonding curve completes.
- Reserved tokens unlock into the pool.
- The token graduates seamlessly to full pool trading or an external AMM.